Milestone Report

Name: Kaden Barthlome
Reporting Period: January 2, 2026 to February 1, 2026
Total Hours Worked on This Project: 25
Total Hours Worked During This Milestone Report: 25
Number of Credits Enrolled: 2

---

1. Milestone Goals vs. Accomplishments

What tasks did you plan to complete during this period?

The tasks I planned to complete during this first phase were setting up the project, understanding the MIDI data structures required, understanding how to map music theory to MIDI data, and building a basic validation pipeline. I also experimented with using Claude Code as a way to accelerate development velocity, treating this milestone partly as a test of how effectively AI-assisted tooling could compress the time between idea and working implementation. This ended up being a significant factor in why the project advanced further than originally scoped. Claude Code handled much of the boilerplate and let me focus on the higher-level architectural decisions, which is why 25 hours produced what might otherwise have taken considerably longer.

What tasks did you actually complete?

Over this milestone, I took the project from an initial commit to a fully functional, tested, and hardened web application. The work breaks down into three phases:

Phase 1

This phase focused on standing up the core application using Vite, React, TypeScript, Tone.js, and @tonejs/midi. I built the foundational types (`MidiComposition`, `Note`, `Track`, `AttemptResult`, `UserPreferences`), implemented the `InputForm` and `AttemptCard` components, added MIDI utilities, and created the OpenRouter service layer.

Along the way, I introduced MIDI velocity normalization via `normalizeVelocity()` to handle models that return velocity as either 0–1 floats or 0–127 integers. After consolidating the app by eliminating the separate `App` component and merging state management into `app/page.tsx`, I added `prompts.md` with prompting tips and updated the model list.

Finally, I built a comprehensive validation layer in `utils/validation.ts` (including `extractJson`, `validatePrefs`, and `validateComposition`) featuring discriminated-union return types, model allowlist enforcement, input size limits, and schema validation of LLM output. I also wrote roughly 750 lines of tests across four Vitest suites (`api-generate.test.ts`, `midiUtils.test.ts`, `route.test.ts`, `scaleUtils.test.ts`). The phase wrapped up with UI polish to the `Page` and `AttemptCard` layouts and the addition of a keyboard shortcut (Ctrl/Cmd+Enter) for form submission.

Phase 2

This phase focused on hardening and standardizing the app's behavior across inputs, security, and configuration. I refactored `InputForm` to normalize all user preferences on submit before sending them to the API, ensuring the backend receives consistent, predictable data.

I added security headers in `next.config.js`, including a Content Security Policy (CSP) along with X-Frame-Options, X-Content-Type-Options, and related protections, while explicitly allowing `blob:` in `script-src` and `worker-src` to preserve Tone.js AudioWorklet compatibility.

To prevent abuse, I implemented in-memory, IP-based rate limiting (10 requests per minute) with periodic cleanup, `Retry-After` handling, and response headers like `X-RateLimit-Remaining` for transparency. I centralized configuration by moving magic numbers into `MIDI_LIMITS` in `constants.ts` and updated all consumers accordingly. I wrapped up by updating the README to reflect the current architecture.

Phase 3

This phase focused on scaling the system and improving overall robustness across MIDI handling, API reliability, and musical correctness. I extracted a reusable `clampNumber` utility, centralized all MIDI range constants, and tightened validation by improving note, tempo, and duration clamping throughout `midiUtils.ts`.

I overhauled error handling in the API route by adding granular responses for specific OpenAI SDK error types, including `APIConnectionTimeoutError`, `APIConnectionError`, and `APIError`, with clear status-specific messaging for 401, 402, 404, 429, and 503, alongside an updated model list.

On the functionality side, I implemented a complete scale system in `utils/scaleUtils.ts`, adding `buildScaleSet`, `snapToScale`, `parseKeyString`, and `isDrumTrack`, with support for nine scale types: major, natural minor, harmonic minor, pentatonic major/minor, blues, dorian, mixolydian, and chromatic.

I integrated scale snapping into the MIDI conversion pipeline so generated notes are quantized to the user's selected scale, while detecting drum tracks to avoid altering percussion. I

expanded the constants layer with `SCALE_ROOTS` and `SCALE_TYPES`. To ensure correctness, I wrote 202 lines of dedicated scale utility tests. Finally, I improved JSON parsing diagnostics by adding tail-logging of the model's JSON output to make it easier to debug truncated or malformed responses.

---

2. Learning and Application

What new knowledge or skills did you gain during this period?

I deepened my understanding of music theory in code by implementing scale theory programmatically, including building pitch class sets from interval arrays, snapping arbitrary MIDI notes to the nearest in-scale pitch, and parsing key signatures from natural language strings. This required a strong grasp of how MIDI note numbers map to pitch classes (`note % 12`) and how scales are defined as interval sets from a root.

On the LLM side, I learned that reliably producing structured JSON output is difficult because models may wrap output in markdown fences, truncate long responses, or return invalid schemas, so I built extraction and validation layers to handle these cases robustly.

Finally, I focused on web app security hardening by implementing Content Security Policy headers, rate limiting, input size validation, and model allowlists. I also gained practical insight into CSP nuances, such as needing `blob:` in `script-src` to support Web Audio API worklets.

What have you applied from your coursework to your project?

I focused on applying what I've learned from my coursework directly to the project. From software engineering fundamentals, I used a modular architecture by cleanly separating validation, utilities, services, and UI components; maintained strong version control discipline through 22 focused commits; leveraged discriminated union types for clearer and safer error handling; and emphasized test-driven validation to ensure reliability.

I also incorporated concepts from my AI in Software Engineering course this semester, particularly prompt engineering, to more effectively guide the AI-assisted development process and improve the quality and consistency of model outputs.

---

3. Challenges and Problem-Solving

What difficulties or mistakes did you encounter?

During development, I ran into several recurring technical challenges that shaped the final architecture and robustness of the app:

- **LLM output inconsistency:** Different models returned JSON in wildly different formats, sometimes wrapping it in markdown code fences, adding explanatory text before or after the payload, or truncating long compositions mid-JSON. Early versions of the app would crash or silently generate broken MIDI files as a result.

- **Velocity format ambiguity:** Some models output velocity as a 0.0–1.0 float (more aligned with Tone.js conventions) while others used 0–127 integers (raw MIDI), resulting in notes that were either inaudibly quiet or heavily clipped until normalization was introduced.

- **CSP regressions:** Once CSP headers were added, Tone.js AudioWorklet playback broke because worklet code is loaded via `blob:` URLs, which a default CSP blocks unless explicitly allowed.

- **Out-of-scale notes:** Models routinely generated out-of-scale notes even when a key signature was specified, producing dissonant results until scale snapping and quantization were integrated into the MIDI pipeline.

- **Persistent Grok failures:** Grok would refuse to generate compositions or return malformed output, making robust, per-attempt error handling essential so a single failed variation could not crash the entire batch.

How did you address or attempt to correct them?

To address the major failure points, I implemented targeted fixes to make the system far more reliable and predictable:

- I built a JSON extraction pipeline around `extractJson()`, which strips markdown fences, finds JSON object boundaries (from the first `{` to the last `}`), and cleanly extracts the payload. I then added `validateComposition()` as a second pass to ensure the output conforms to the expected schema, along with JSON tail-logging to help diagnose truncated responses.

- To solve velocity inconsistency across models, I wrote `normalizeVelocity()` to detect whether a value is in the 0–1 or 0–127 range and normalize accordingly, treating values greater than 1 as raw MIDI velocity and scaling them down into the 0–1 range.

- To fix the CSP playback regression, I added `blob:` to both the `script-src` and `worker-src` directives in `next.config.js`, restoring Tone.js AudioWorklet

functionality while keeping the rest of the policy restrictive.

- For musical correctness, I added post-generation scale snapping by building out `scaleUtils.ts`, which snaps every generated note to the nearest pitch in the user's selected scale after generation, acting as a safety net regardless of what the LLM produces. I also added an `isDrumTrack()` heuristic to detect and skip percussion tracks so scale snapping would not corrupt drum patterns.

- Finally, I implemented granular error handling by wrapping the OpenAI SDK call with specific `catch` branches for `APIConnectionTimeoutError`, `APIConnectionError`, and `APIError` with status-specific responses, allowing the UI to surface clear, actionable messages instead of generic failures.

Describe an interesting or challenging problem you faced and how you approached solving it.

The most interesting problem I tackled was building the scale-snapping system, and the challenge ended up being multi-layered.

First, I learned quickly that I could not rely on the LLM to apply music theory consistently. Even when prompted for something like "C Minor," it might output notes in C Major, D Dorian, or an essentially random chromatic pattern. That meant the scale logic had to be enforced programmatically as a post-processing step rather than treated as a model guarantee.

From there, the snapping algorithm needed to map any out-of-scale note to the nearest in-scale pitch, so I implemented a bidirectional search that checks offsets from ±1 up to ±6 semitones relative to the target pitch class, with a tie-break rule that favors the lower pitch. Doing this correctly required thinking in modular arithmetic over pitch classes (mod 12) rather than treating notes as purely linear integers.

I also had to account for percussion, since drum tracks use MIDI note numbers to represent specific drum hits rather than pitched notes. Snapping those to a musical scale would completely destroy the rhythm, so I added an `isDrumTrack()` heuristic to detect and skip percussion tracks.

Finally, I needed to reliably interpret the key string coming back from the model (for example, "F# Minor" or "Bb Dorian"), which required parsing natural language into a numeric root and a scale type while handling enharmonic spellings and common abbreviations.

---

4. Support and Improvement

What skills or areas would you like help with to perform better on this project?

I could use help with production deployment, which involves moving the app from a development setup into a real production environment by deploying on Vercel, tightening up environment and secret management, and replacing the current in-memory rate limiting with a Redis-backed approach so limits are consistent, scalable, and reliable across multiple server instances. Any tips on this would be greatly appreciated.

---

5. Self-Evaluation

How would you rate your performance for this milestone, and why?

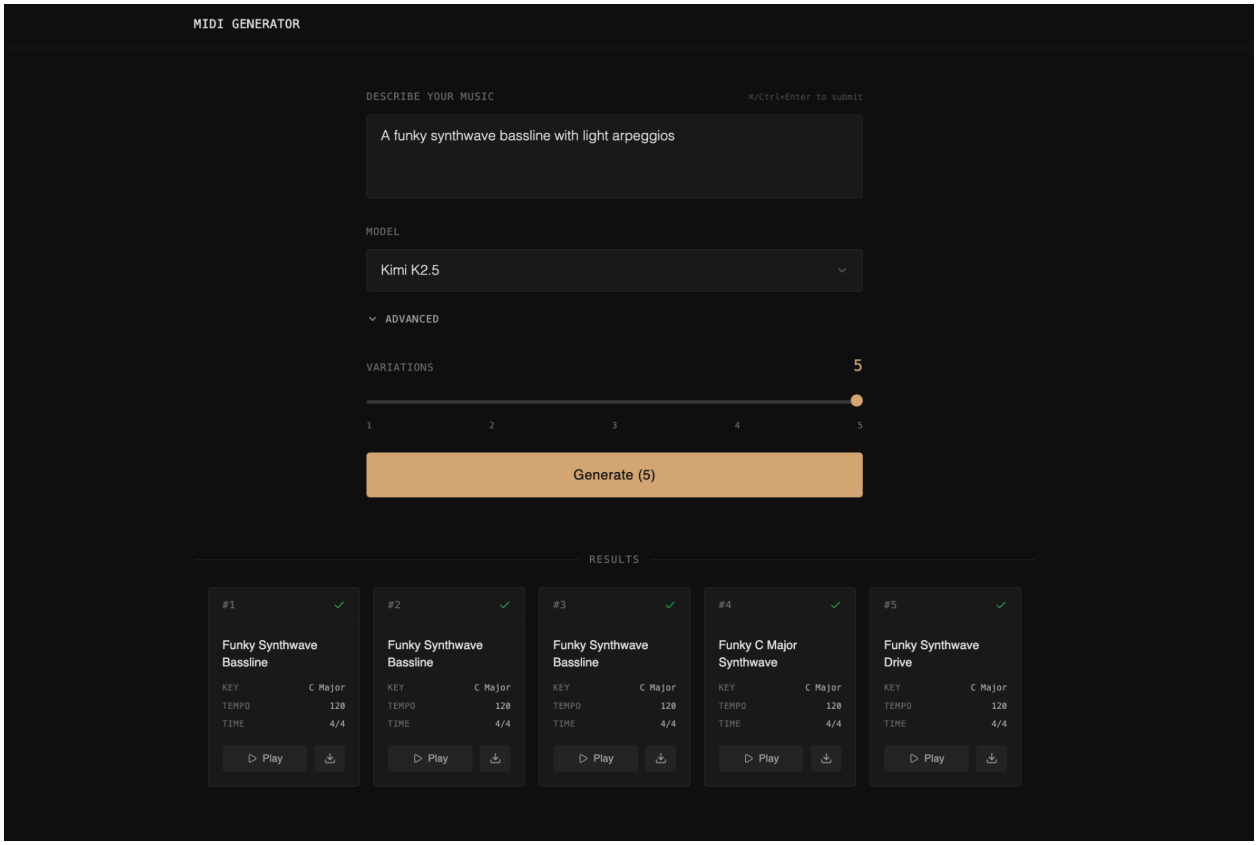
I would rate this milestone a **9/10**, holding back the final point because authentication is still missing. I explicitly called this out in my January 23 journal as a next step, but I did not get to it within the scope of this milestone. While the server-side API key is protected, the app still lacks user accounts and saved compositions, which is a meaningful gap that I plan to close in the next milestone.

What went well was the validation architecture, the scale system design, security hardening, strong testing discipline, and maintaining focused commits throughout the work. If I were to do it differently, I would start authentication earlier in the milestone and set firmer scope boundaries before expanding into additional features.

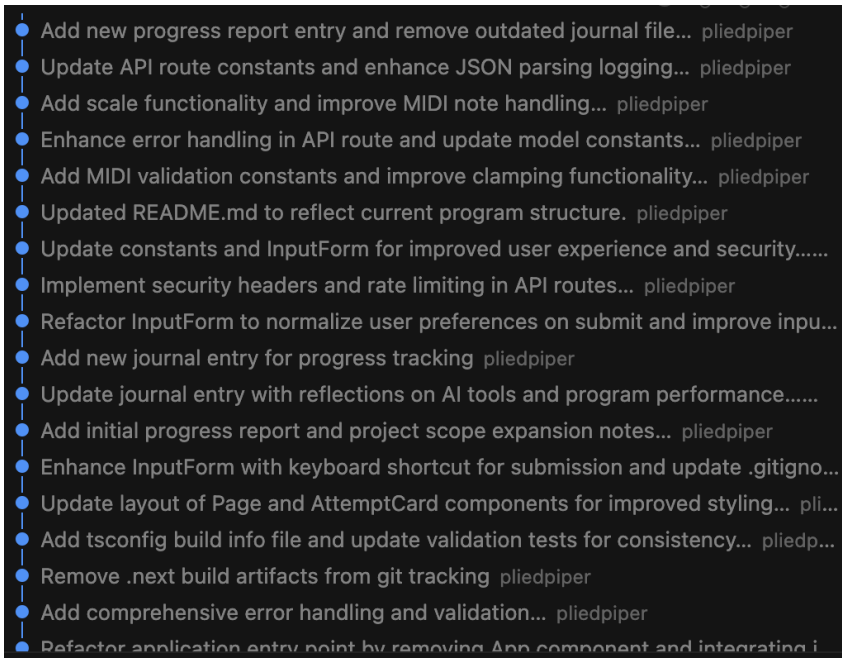
---

6. Evidence of Work

Program Dashboard



Commit History



---

7. Additional Comments

This project has been one of the more rewarding things I've worked on in school because it sits at the intersection of music and software, two things I actually care about. I've been using the generator throughout the week to come up with design ideas for electronic music, and even in its current state it's a genuinely useful tool for inspiration. The MIDI files still need tweaking, but having something spit out a bassline or arpeggio pattern that I didn't think of on my own is surprisingly valuable creatively.

One thing that surprised me was how much of the work ended up being about defending against bad LLM output rather than building features. I expected the hard part to be audio playback or the UI, but in practice the real challenge was that every model returns JSON differently, ignores instructions in its own way, and can't be trusted to stay in key. A huge chunk of the codebase (validation, scale snapping, JSON extraction) exists purely because you can't treat an LLM like a reliable API. That was a useful lesson about building on top of non-deterministic systems.

I also found that newer models produce noticeably better music. Gemini 3 Pro and Gemini 3 Flash, and the newly released Kimi 2.5, have been the standouts. They respect the schema more consistently and generate compositions that actually sound intentional. GPT-5.2 is solid. Grok frequently fails to generate anything at all, which pushed me to build better error handling

early on. That variation across models reinforced why the multi-model selector and robust fallback behavior matter.

On the tooling side, this project doubled as an experiment in AI-assisted development. I used Claude Code extensively throughout the milestone, and it meaningfully changed how I worked. The biggest wins were in boilerplate generation, test scaffolding, and rapid iteration on validation logic. Tasks where I knew exactly what I wanted but would have spent time on syntax and wiring. It also helped me move faster on unfamiliar territory like CSP headers and Tone.js AudioWorklet quirks, where I could describe the problem and get pointed toward solutions I wouldn't have found as quickly on my own. That said, it wasn't frictionless. Claude Code occasionally introduced subtle bugs or made assumptions about project structure that required careful review, and I learned quickly that treating its output as a first draft rather than finished code was essential. In a way, the experience mirrored the core challenge of the project itself: building reliably on top of systems that are helpful but non-deterministic. The difference is that Claude Code's mistakes are easier to catch in a code review than an LLM's bad JSON is to catch at runtime, which is why so much of the app's architecture exists to defend against the latter.

Looking ahead, I want to get authentication and saved compositions working so users can come back to things they liked. I'd also like to explore a piano roll visualization. Right now you hear the output but can't see it, which makes it harder to evaluate or tweak. The scale snapping was a big step toward output quality, but there's a lot more room for post-processing like chord voicing cleanup and rhythm quantization.